INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE DO NORTE
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
SISTEMAS OPERACIONAIS · 2026

LABORATÓRIO 2

Da chamada de sistema ao módulo de kernel

As quatro travessias da fronteira entre modo usuário e modo núcleo

`read() → 0` · `SIGSEGV → 139` · `SIGILL → 132` · `insmod → comm=insmod`

UNIDADE	1 — Introdução
TEMA	Estrutura do SO: modo de execução, privilégio e capacidades
LINGUAGEM	C (usuário e kernel) e Make
AMBIENTE	Container `so-ifrn` da disciplina
ENTREGA	Código-fonte + relatório PDF + defesa oral
REFERÊNCIA	Tanenbaum, *Sistemas Operacionais Modernos*, §1.6, p. 35 e §1.7, p. 43

ROTEIRO DO LABORATÓRIO

Parte A Como um programa em modo usuário chega ao núcleo
- A1 Chamada de sistema — o programa pede · `chamada.c`
- A2 Exceção — o programa erra · `excecao.c`
- A3 Instrução privilegiada — o programa tenta sozinho · `privilegio.c`

Parte B Como colocar código seu dentro do núcleo · `ola.c`
- 1–10 Kbuild, `vermagic`, as duas barreiras e a carga real

Parte C Síntese, entregáveis e questões

O que você vai fazer

Esta prática tem três partes, e as duas primeiras existem para que a terceira faça sentido.

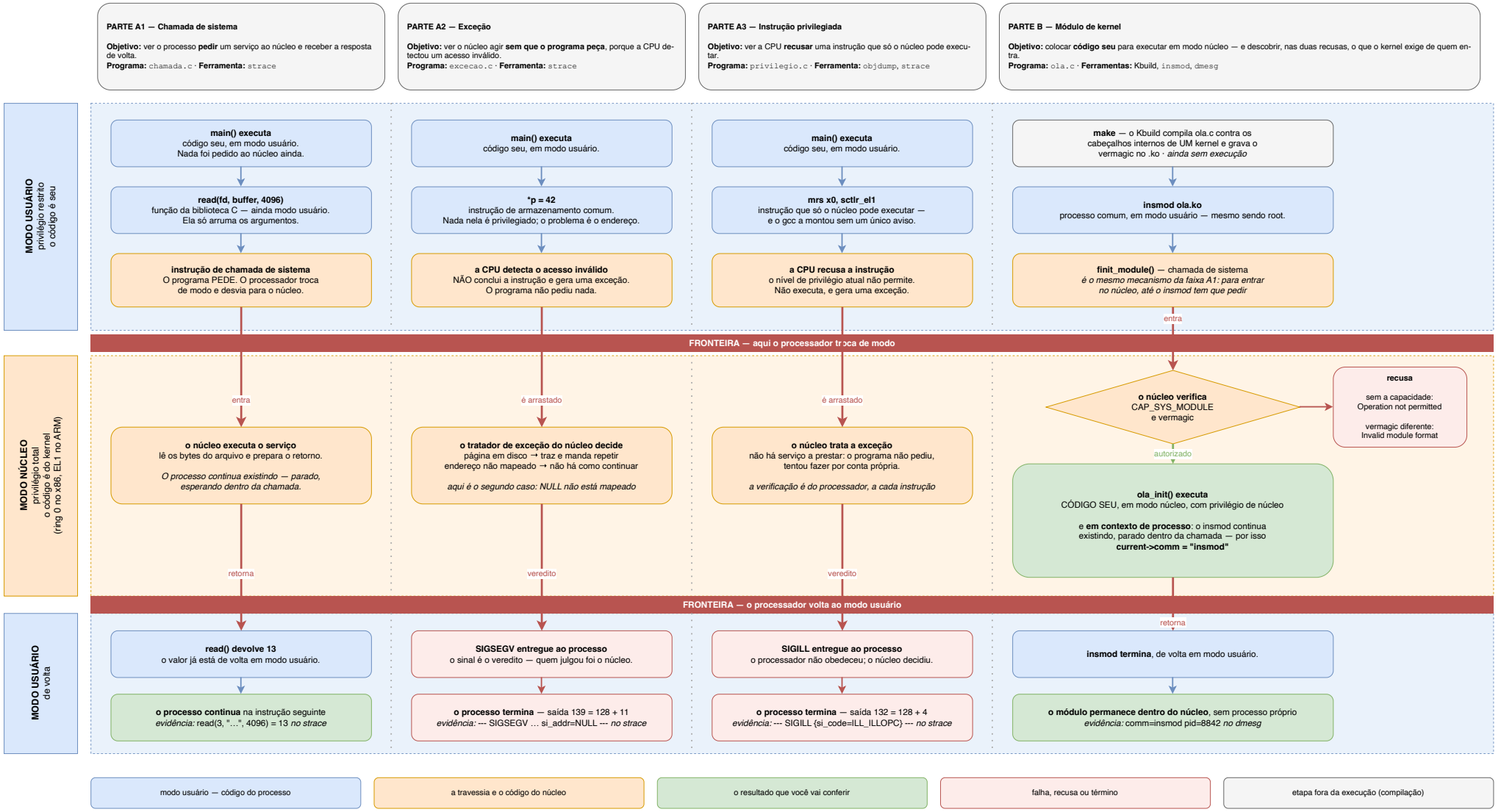
Parte A — como um programa em modo usuário chega ao núcleo. Três programas curtos. O primeiro **pede** um serviço ao núcleo com uma chamada de sistema. O segundo **não pede nada** e mesmo assim o núcleo entra em ação, porque o processador detectou um acesso inválido. O terceiro **tenta fazer sozinho** o que só o núcleo pode fazer — e o processador recusa. Três maneiras de chegar ao mesmo lugar, observadas por fora.

Parte B — como colocar código seu dentro do núcleo. Escrever, compilar e tentar carregar um **módulo de kernel**: 24 linhas de C que não têm `main()`, não viram executável e não podem usar a biblioteca C. Você vai esbarrar em duas barreiras diferentes antes de conseguir.

Parte C — o que a fronteira é, afinal. A síntese das duas partes.

Aula 03 — As quatro travessias da fronteira entre modo usuário e modo núcleo

Cada faixa vertical é um experimento do roteiro. As três primeiras mostram como o núcleo passa a executar por causa do seu programa; a quarta coloca código seu do outro lado.



Nas quatro faixas o destino é o mesmo: código do núcleo executando em modo privilegiado. O que muda é quem começou a travessia — o programa, o processador ou você.

As quatro travessias desta prática

As quatro faixas verticais são os quatro experimentos, e cada passo deste roteiro traz de novo a sua faixa, ampliada. Se as figuras não aparecerem no seu leitor de Markdown, abra os arquivos `fluxo-*.svg` desta pasta — ou os `.drawio.png`, que são a mesma coisa em imagem.

A pergunta que amarra tudo

Todo programa que você já escreveu executa em **modo usuário**. Nesse modo o processador recusa instruções privilegiadas e recusa acesso à memória de outros processos. Ainda assim, o seu programa lê arquivos, cria processos e escreve na tela — coisas que só o núcleo pode fazer de verdade.

A sequência que esta prática constrói é esta:

```
1. Meu programa começa executando em modo usuário.

2. Ele precisa de um serviço do sistema operacional.

3. Uma chamada de sistema faz o processador entrar em modo núcleo.

4. O núcleo executa o serviço pedido.

5. O controle volta para o programa, em modo usuário.

6. Uma exceção também faz o processador transferir
   o controle para o núcleo — sem que o programa peça.

7. E se eu escrever, no meu programa, a instrução
   privilegiada que o núcleo usaria? O processador
   recusa executá-la, e isso também vira uma exceção.

8. Portanto o processo não "vira núcleo". Existem momentos
   em que o mesmo processo está associado à execução de
   código do núcleo — mas quem executa código privilegiado
   é sempre o núcleo, nunca o meu programa.

9. Daí a próxima pergunta: e se o código do núcleo
   que executa nesses momentos for MEU?

10. É isso que um módulo de kernel permite investigar.
```

Os passos 1 a 7 são a **Parte A**. Os passos 9 e 10 são a **Parte B**.

> **COMO ESTE ROTEIRO FUNCIONA**
>
> **Vários passos aqui falham de propósito.** Não são erros do roteiro nem da sua máquina: cada falha é uma pergunta, e o passo seguinte é a resposta. Um dos programas da Parte A **morre de propósito**.
>
> Antes de rodar cada comando marcado com 🔮 **Preveja**, escreva num papel o que você acha que vai acontecer. Depois rode. **A distância entre o que você previu e o que apareceu é o conteúdo desta prática** — e algumas dessas previsões viram entregável no fim.
>
> Não pule para a seção de erros comuns no primeiro tropeço.

Vocabulário: três coisas que costumam ser confundidas

Fixe estes três antes de começar, porque o roteiro inteiro depende deles.

Modo de execução é um estado do **processador**. Em modo usuário certas instruções e certos endereços são recusados pelo hardware; em modo núcleo, não. O modo vale para o **trecho de código que está exe-**

cutando agora, não para o programa como um todo.

> **ATENÇÃO**
>
> ⚠ **Não diga "esta aplicação roda em modo usuário *ou* em modo núcleo"**, como se fossem dois estados permanentes do processo. O correto é: *um processo normalmente executa o próprio código em modo usuário; durante a execução, determinadas situações fazem o processador transferir temporariamente o controle para código do núcleo, que executa em modo privilegiado.*

`root` é outra coisa. `root` é uma **identidade**, verificada pelo sistema operacional por software, que dá direito a operações que o SO restringe. Modo usuário/núcleo é **propriedade do contexto de execução no processador**. Um processo do `root` executa o próprio código em modo usuário exatamente como o seu — ele tem mais direitos, não mais poder de máquina. A Parte B mede essa diferença.

Sobre "ring 0" e "ring 3". Essa numeração é a nomenclatura da arquitetura **x86**, que define quatro anéis (dos quais os SOs de uso geral usam dois). **Não é uma definição universal.** O ARM, que é o processador do container desta disciplina, chama os mesmos níveis de **EL0** (aplicação) e **EL1** (núcleo), e ainda tem EL2 e EL3 para outros fins. Quando este roteiro precisa do conceito, ele diz **modo usuário** e **modo núcleo**; "ring 0" aparece só quando o texto está falando especificamente de x86.

Os três mecanismos de entrada no núcleo

São relacionados — todos desviam a execução para um ponto de entrada do núcleo, em modo privilegiado — mas são **conceitualmente diferentes**, e a diferença está em *quem começou*:

mecanismo	origem	é voluntário?	exemplo
chamada de sistema	o próprio programa, por instrução dedicada	**sim** — o programa pediu	`read()`, `fork()` — **Parte A1**
exceção	o processador, ao executar uma instrução do programa	**não** — o programa errou, ou tentou o que não pode	acesso inválido — **Parte A2** · instrução privilegiada — **Parte A3** · divisão por zero, falta de página
interrupção	um dispositivo **externo**, de forma assíncrona	**não** — nem tem relação com a instrução em curso	temporizador, teclado, disco, rede

A **interrupção** é o terceiro mecanismo e não tem experimento próprio aqui: ela não é provocada pelo seu programa, e sim pelo hardware, a qualquer momento. É o que permite ao SO retomar o processador de um processo que não pede nada — e é o assunto da Unidade 2, quando o temporizador virar a base do escalonamento preemptivo.

Um erro comum é usar "interrupção" como nome genérico dos três. Chamada de sistema é pedido; exceção é falta detectada na instrução em curso; interrupção vem de fora.

Repare que os passos **A2 e A3 são os dois o mecanismo de exceção** — e é exatamente por isso que valem dois experimentos. A exceção não serve só para erro de endereço: ela é o mecanismo geral com que o processador interrompe uma instrução que não pode concluir, seja porque o endereço não é seu (A2), seja porque a instrução não é sua (A3).

PARTE A — Como um programa em modo usuário chega ao núcleo

Passo 0 — Entrar no laboratório

A partir da pasta `ambiente/`:

```
./so.sh                # macOS e Linux
.\so.ps1               # Windows (PowerShell)
```

Dentro do container:

```
cd /so/Aula03/codigo
ls
```

```
Makefile  chamada.c  excecao.c  ola.c  privilegio.c
```

É aqui que você trabalha o tempo todo. São os quatro programas desta prática, na ordem em que você vai usá-los — `chamada.c` (A1), `excecao.c` (A2), `privilegio.c` (A3) e `ola.c` (Parte B) — mais o `Makefile`, que só a Parte B usa.

A pasta de cima, `/so/Aula03`, guarda este roteiro, os diagramas e a subpasta `fronteira/`. A tabela do fim lista tudo.

Confirme onde você está:

```
uname -m
uname -r
```

```
aarch64
6.12.76-linuxkit
```

> Este roteiro foi medido em **`aarch64`** (Mac com Apple Silicon). Se aparecer `x86_64`, tudo funciona igual — os números e os nomes de sinal podem mudar, e o roteiro avisa onde. Anote o que apareceu.

Passo A1 — A travessia pedida: uma chamada de sistema

O arquivo `chamada.c` faz uma coisa só: **lê 4096 bytes de um arquivo**. O que interessa não é o `read()` — é o caminho que a execução percorre por causa dele.

```
printf("[usuario] 1. codigo do processo; nada pedido ao nucleo ainda\n");

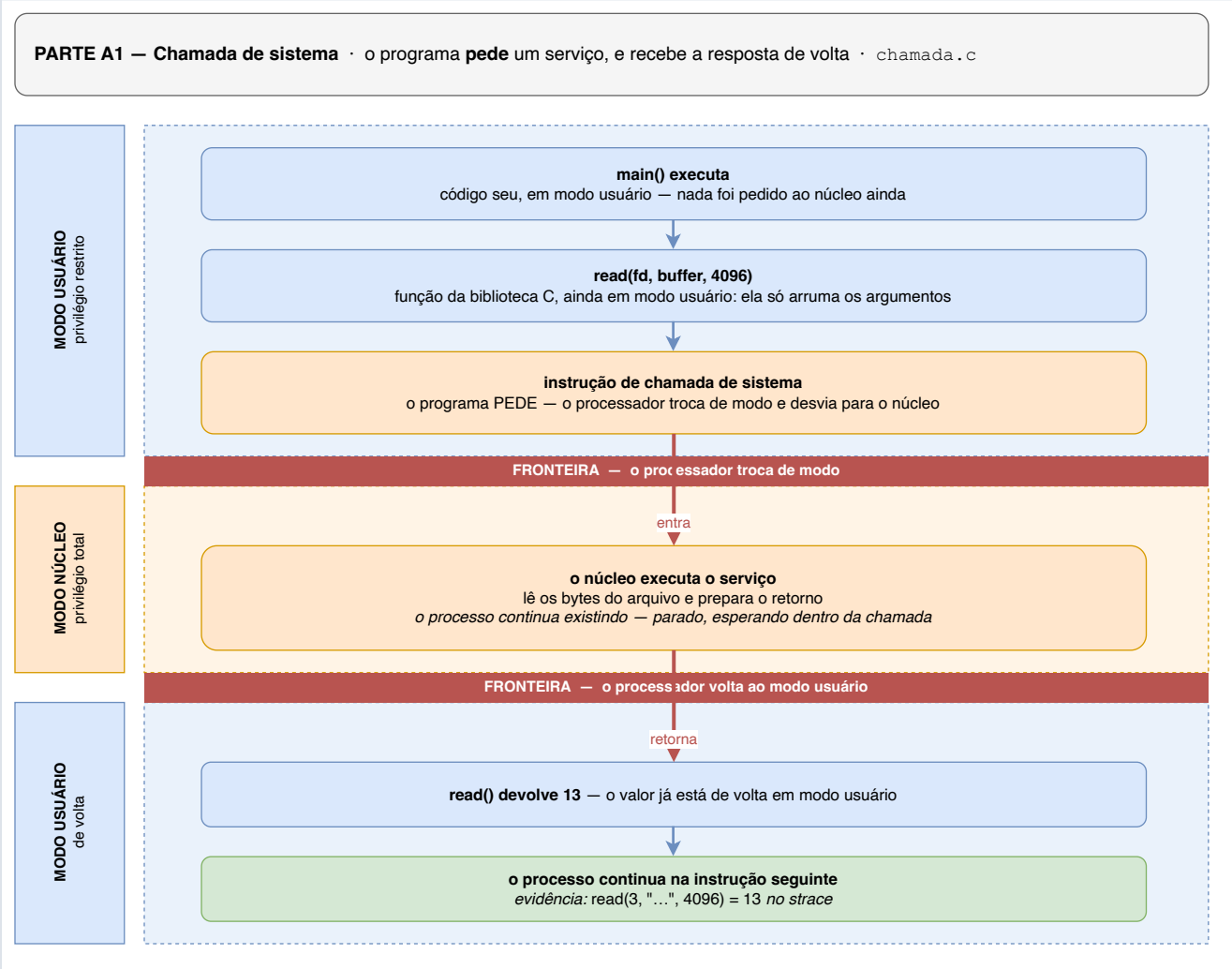
fd = open(caminho, O_RDONLY);

printf("[usuario] 2. pedindo %d bytes ao nucleo com read()\n", TAM);
n = read(fd, buffer, TAM);                          /* ❶ pela biblioteca C */
printf("[usuario] 3. de volta em modo usuario; read() devolveu %zd bytes\n", n);

lseek(fd, 0, SEEK_SET);
n = syscall(SYS_read, fd, buffer, (size_t)TAM);   /* ❷ sem a biblioteca C */
printf("[usuario] 4. mesmo servico sem a funcao read(): %zd bytes\n", n);
```

O mesmo serviço é pedido **duas vezes**, por dois caminhos: ❶ pela função `read()` da biblioteca C e ❷ por `syscall(SYS_read, …)`, que dispensa essa função. Guarde isso — é o ponto do passo.

O percurso que você vai observar



Fluxo do Passo A1 — chamada de sistema

Em texto, o mesmo percurso:

```
        código do processo
                ↓
          modo usuário
                ↓
        chamada de sistema
                ↓
          modo núcleo
                ↓
   o núcleo executa o serviço
                ↓
    retorno ao modo usuário
                ↓
    continuação do processo
```

PREVEJA ANTES DE SEGUIR

O `strace` lista todas as chamadas de sistema que um processo faz.

1. Quantas linhas de `read` você espera ver na região do **seu** código — uma, duas, ou nenhuma?
2. A chamada feita por `read()` e a feita por `syscall(SYS_read, …)` vão aparecer **iguais** ou **diferentes** no `strace`? Por quê?

Compile e execute:

```
gcc -Wall -Wextra -O0 -o chamada chamada.c
./chamada
```

```
[usuario] 1. codigo do processo; nada pedido ao nucleo ainda
[usuario] 2. pedindo 4096 bytes ao nucleo com read()
[usuario] 3. de volta em modo usuario; read() devolveu 13 bytes
[usuario] 4. mesmo servico sem a funcao read(): 13 bytes
[usuario] 5. conteudo lido de /etc/hostname: "8db58470beee"
```

(O conteúdo é o nome do container, então o seu vai ser outro. São 13 bytes: 12 do nome mais o `\n`.)

Agora **torne a travessia visível**. O `> /dev/null` joga fora a saída do programa para sobrar só o rastro das chamadas:

```
strace -e trace=openat,read,lseek,close ./chamada > /dev/null
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\360\206\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
read(3, "761dee9f9ee5\n", 4096)         = 13
lseek(3, 0, SEEK_SET)                   = 0
read(3, "761dee9f9ee5\n", 4096)         = 13
close(3)                                = 0
+++ exited with 0 +++
```

(O filtro `-e trace=` esconde as demais; para ver tudo, rode `strace ./chamada`.)

Descoberta 1 — cada linha dessas é uma ida e uma volta

Cada linha do `strace` é uma travessia completa: o processo, em modo usuário, emitiu a instrução de chamada de sistema; o processador trocou de modo e desviou para o ponto de entrada do núcleo; o núcleo executou o serviço; o processador voltou ao modo usuário; e a execução continuou na instrução seguinte. **O `= 13` no fim da linha é o valor de retorno já de volta do outro lado.**

Repare que o programa **não desviou o fluxo**. Do ponto de vista do código C, `read()` foi uma chamada de função como qualquer outra: chamou, recebeu um número, seguiu adiante. A troca de modo aconteceu por baixo.

Descoberta 2 — as duas leituras produziram linhas idênticas

```
read(3, "761dee9f9ee5\n", 4096)         = 13      ← veio de read()
lseek(3, 0, SEEK_SET)                   = 0
read(3, "761dee9f9ee5\n", 4096)         = 13      ← veio de syscall(SYS_read, …)
```

Byte por byte, a mesma linha. O `strace` não consegue distinguir as duas porque, **do lado do núcleo, não há nada para distinguir**: chegou o mesmo pedido, com os mesmos argumentos.

É por isso que dizer "*a função `read()` roda no kernel*" está errado. `read()` é uma função da biblioteca C, compilada dentro do seu processo, executando em modo usuário. O que ela faz é arrumar os argumentos nos registradores combinados e emitir a instrução de chamada de sistema. **A função é sua; o serviço é do núcleo.** A prova é que dá para obter o serviço sem a função — foi o que a linha ❷ fez.

A formulação correta é: *o processo pediu um serviço ao núcleo por meio de uma chamada de sistema; o serviço foi executado por código do núcleo, em modo privilegiado; o resultado foi devolvido ao processo, que voltou a executar em modo usuário.*

Descoberta 3 — as linhas antes do seu open também são travessias

`openat` do `ld.so.cache`, `openat` da `libc.so.6`, o `read` de 832 bytes com `\177ELF` visível: nada disso está no seu código. É o **carregador dinâmico** montando o processo antes do `main()` começar. Cada uma dessas linhas também é uma ida e volta ao núcleo.

E as próprias linhas `[usuario]` que apareceram na tela são travessias: `printf` acaba chamando `write()`. Rode `strace -c ./chamada > /dev/null` e conte — dá **40 chamadas** para um programa de vinte linhas úteis.

> ***Quer medir o preço dessa travessia em nanossegundos e deduzir o tamanho do buffer da biblioteca C contando chamadas?*** *Há um laboratório à parte, opcional, na subpasta `fronteira/`, que aprofunda exatamente este passo. Aqui basta ter visto o mecanismo.*

Passo A2 — A travessia involuntária: uma exceção

No passo A1 o programa **pediu**. Agora um programa que não pede nada.

O arquivo `excecao.c` inteiro:

```c
#include <stdio.h>

int main(void)
{
    int *p = NULL;

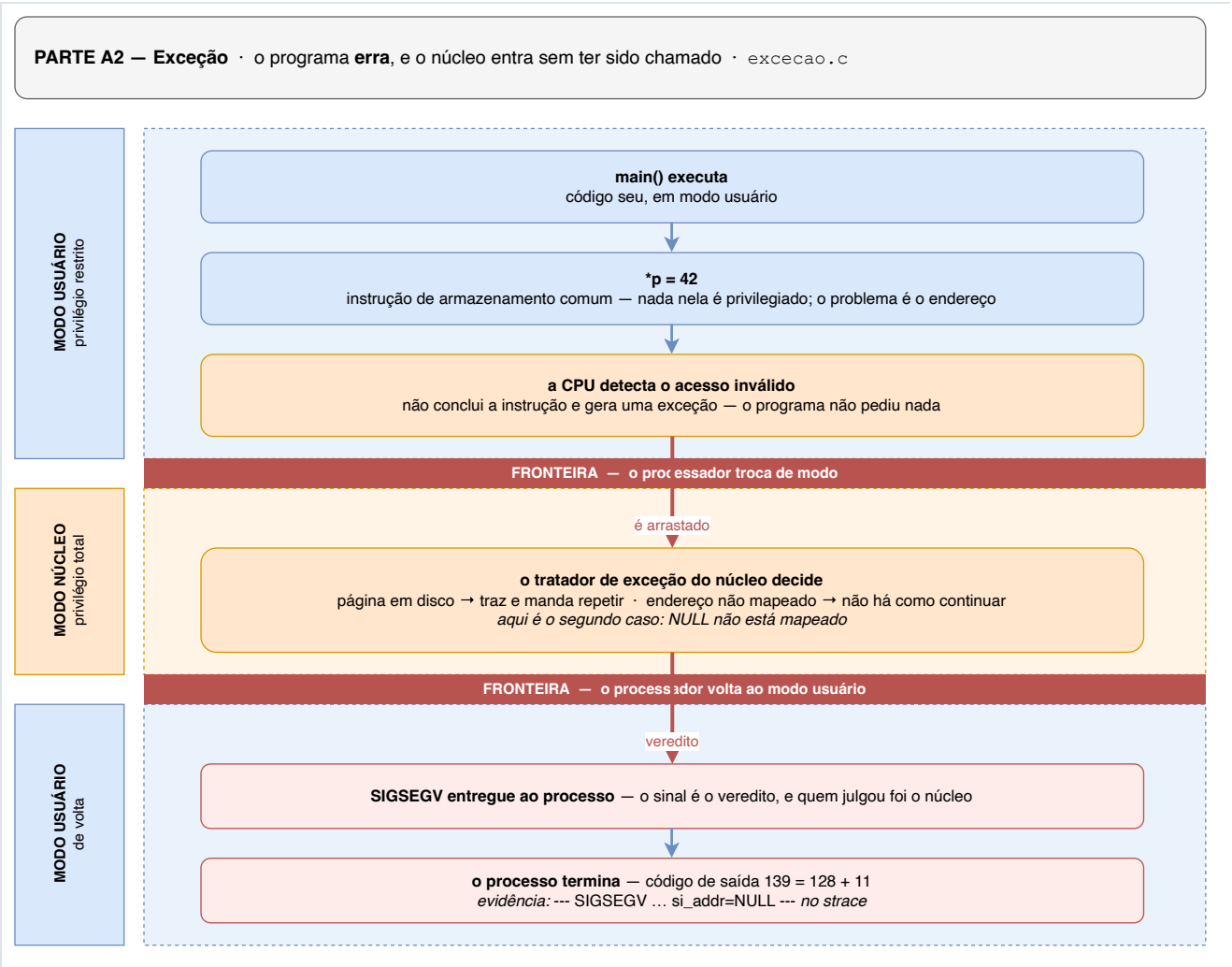
    printf("Antes do acesso\n");
    fflush(stdout);

    *p = 42;

    printf("Depois do acesso\n");
    return 0;
}
```

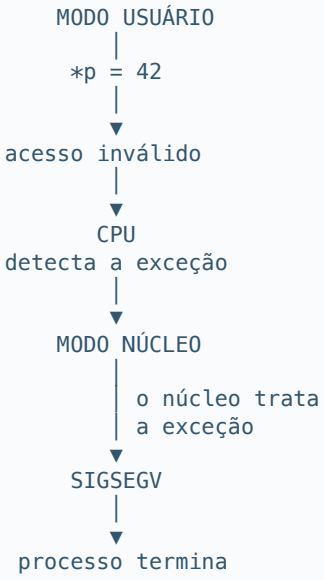
`*p = 42` escreve o valor 42 no endereço guardado em `p`. Não há nada de privilegiado nessa operação: escrever na memória é a coisa mais comum que um programa faz. O problema é o endereço — `NULL`, que no Linux deste laboratório não corresponde a nenhuma página válida do processo.

O percurso que você vai observar



Fluxo do Passo A2 — exceção por acesso inválido

Em texto, o mesmo percurso:



PREVEJA ANTES DE SEGUIR

Escreva **três** respostas, separadas:

1. O `gcc -Wall -Wextra` **compila** este arquivo? Com erro, com aviso, ou em silêncio?
2. As duas linhas — `Antes do acesso` e `Depois do acesso` — aparecem?
3. O programa chega a fazer alguma **chamada de sistema** para pedir que o erro seja tratado?

Compile:

```
gcc -Wall -Wextra -O0 -o excecao excecao.c
echo "gcc saiu com: $?"
```

```
gcc saiu com: 0
```

Nenhum aviso. Com `-Wall -Wextra` ligados, o compilador não teve uma objeção. Confira que a instrução gerada é banal:

```
objdump -d excecao | grep -A14 "<main>:"
```

```
00000000000007d8 <main>:
 7d8:   a9be7bfd    stp x29, x30, [sp, #-32]!
 7dc:   910003fd    mov x29, sp
 7e0:   f9000fff    str xzr, [sp, #24]
 7e4:   90000000    adrp    x0, 0 <__abi_tag-0x278>
 7e8:   91214000    add x0, x0, #0x850
 7ec:   97ffffad    bl  6a0 <puts@plt>
 7f0:   f00000e0    adrp    x0, 1f000 <__FRAME_END__+0x1e6a0>
 7f4:   f947f000    ldr x0, [x0, #4064]
 7f8:   f9400000    ldr x0, [x0]
 7fc:   97ffffad    bl  6b0 <fflush@plt>
 800:   f9400fe0    ldr x0, [sp, #24]
 804:   52800541    mov w1, #0x2a                   // #42
 808:   b9000001    str w1, [x0]
 80c:   90000000    adrp    x0, 0 <__abi_tag-0x278>
```

O `mov w1, #0x2a` carrega o 42 e o `str w1, [x0]` no endereço `808` faz a escrita. **É uma instrução de armazenamento comum** — a mesma que aparece em qualquer atribuição do seu código. Nada nela é privilegiado, e nada nela é ilegal *em si*. O que decide é o endereço que estiver em `x0` **na hora da execução**.

(Se você reparou que o primeiro `printf` virou `puts`: sim, o compilador troca um `printf` de string constante terminada em `\n` pelo `puts`, que é mais barato. Não muda o experimento.)

Execute:

```
./excecao
echo "codigo de saida: $?"
```

```
Antes do acesso
Segmentation fault (core dumped)
codigo de saida: 139
```

O `139` não é aleatório: um processo morto por sinal sai com **128 + número do sinal**.

```
kill -l 11
```

```
SEGV
```

139 − 128 = 11 = SIGSEGV.

A evidência de que o núcleo entrou sem ser chamado

Rode sob o `strace`, filtrando só as escritas:

```
strace -e trace=write ./excecao
```

```
write(1, "Antes do acesso\n", 16)       = 16
Antes do acesso
--- SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_MAPERR, si_addr=NULL} ---
+++ killed by SIGSEGV +++
```

Leia com atenção o que **não** está aí. Há **uma** chamada de sistema — a `write` do `printf` — e ela é o passo A1 acontecendo de novo. Depois dela **não há nenhuma outra**. O programa não pediu nada ao núcleo a respeito do erro. Ainda assim o núcleo agiu:

- `si_signo=SIGSEGV` — o sinal escolhido pelo núcleo;
- `si_code=SEGV_MAPERR` — o motivo: endereço **não mapeado** para este processo;
- `si_addr=NULL` — o endereço que causou a falta. O núcleo sabe **qual** endereço foi porque o processador o registrou ao gerar a exceção.

O que você acabou de descobrir

O programa não convocou o núcleo; o processador convocou. Enquanto executava a instrução `str w1, [x0]` em modo usuário, o processador consultou o mapeamento de memória do processo, não encontrou o endereço, e **não concluiu a instrução**. Ele gerou uma exceção: trocou para modo núcleo e desviou para o tratador que o SO instalou.

O `SIGSEGV` é o resultado visível para o processo, não o mecanismo. A ordem é esta: a CPU detecta a violação → transfere o controle para o núcleo → **o núcleo decide o que fazer** → uma das decisões possíveis é entregar `SIGSEGV` ao processo. O sinal é o veredito, e quem julgou foi o núcleo.

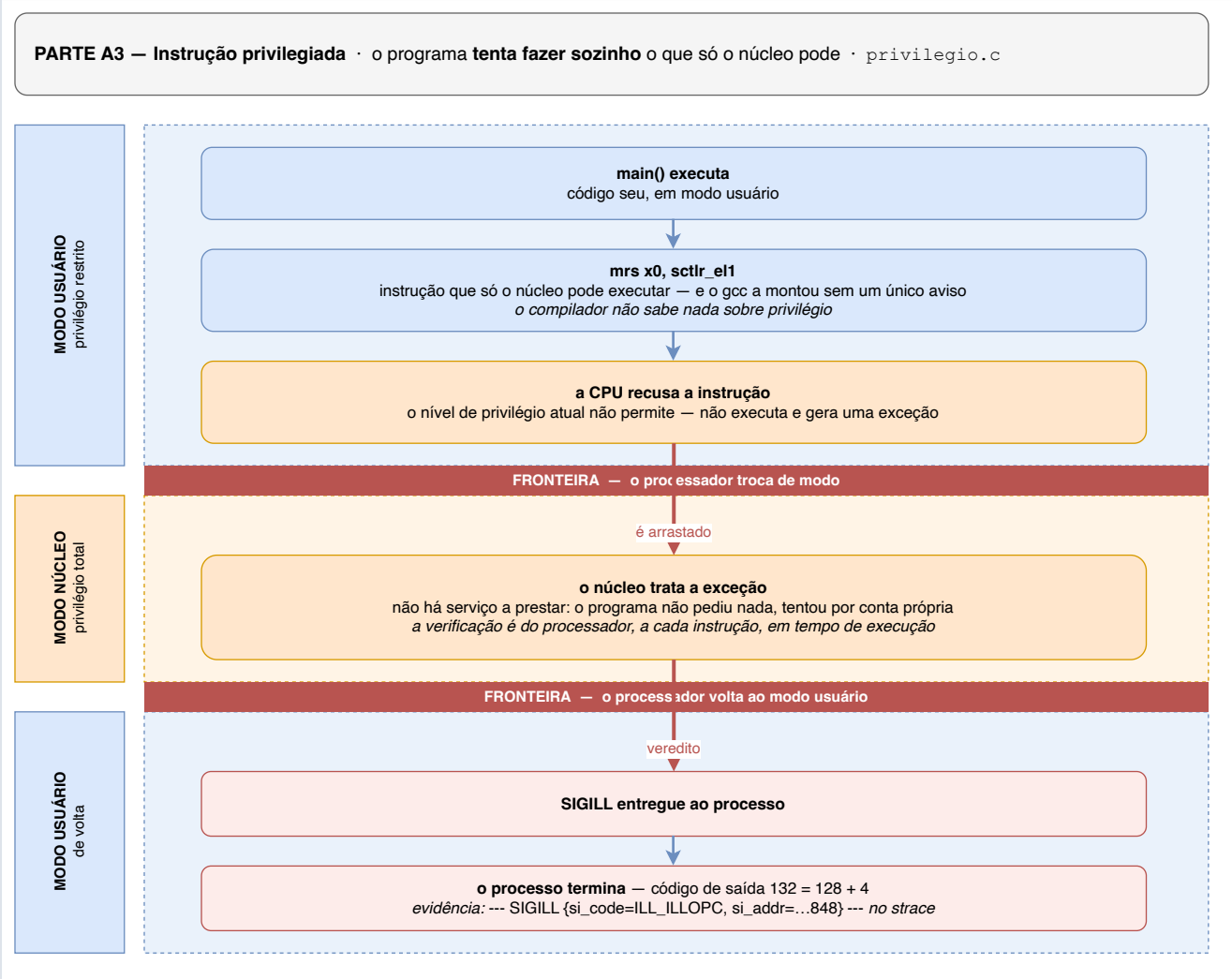
E a decisão nem sempre é matar. O mesmo mecanismo — exceção por endereço que a MMU não resolve — é o que sustenta a memória virtual: quando a página existe mas está em disco, o núcleo traz a página, corrige o mapeamento e **manda o processo repetir a instrução**, sem que ele perceba. É a *falta de página* da Unidade 3. Aqui a página não existe mesmo, e por isso o veredito foi o sinal.

> **ATENÇÃO**
>
> Este roteiro para por aqui na questão da memória, de propósito. O que importa agora é o mecanismo de entrada no núcleo, não o gerenciamento de memória.
>
> ⚠ **Este resultado é do Linux desta prática.** Que o endereço `NULL` seja inválido é uma **decisão do sistema operacional**, que deixa a primeira página do espaço de endereçamento deliberadamente sem mapeamento justamente para transformar o bug clássico do ponteiro nulo numa falha barulhenta. Não é uma lei da arquitetura: em outros sistemas e em ambientes sem proteção de memória (microcontroladores, por exemplo), o endereço 0 pode ser perfeitamente gravável e a escrita simplesmente acontece — silenciosa e destrutiva. Em `x86_64` com Linux o resultado é o mesmo que você viu; o nome do sinal e o código de saída 139 se mantêm.

Passo A3 — A travessia recusada: uma instrução privilegiada

Nos dois passos anteriores o programa **pediu** (A1) e **errou** (A2). Falta a terceira possibilidade, e é a mais tentadora de todas: **fazer sozinho**.

O raciocínio é natural. O núcleo consegue executar instruções privilegiadas porque o processador está em modo núcleo quando ele roda. Mas o meu programa também é um monte de instruções. Então: **e se eu simplesmente escrever a instrução privilegiada dentro do meu código?** O compilador aceita? O processador executa?



Fluxo do Passo A3 — instrução privilegiada

O arquivo `privilegio.c` faz exatamente isso. O miolo são três linhas:

```
printf("antes:  processo em modo usuario, prestes a %s\n", ALVO);
fflush(stdout);

__asm__ volatile("mrs %0, sctlr_el1" : "=r"(valor));

printf("depois: a instrucao privilegiada passou, valor=0x%lx\n", valor);
```

`SCTLR_EL1` é o **registrador de controle do sistema** do ARM: ele liga e desliga a MMU, os caches e a checagem de alinhamento — ou seja, controla o próprio mecanismo que fez o Passo A2 falhar. O sufixo `_EL1` diz o nível de execução mínimo para tocar nele, e **um processo de usuário roda em EL0**. Em `x86_64` o arquivo compila o outro ramo, com `cli`, que desliga as interrupções mascaráveis; o desfecho é o mesmo, com outro nome de sinal — veja a nota no fim deste passo.

> **PREVEJA ANTES DE SEGUIR**
>
> Escreva **três** respostas, separadas:
>
> 1. O `gcc -Wall -Wextra` **compila** este arquivo? Com erro, com aviso, ou em silêncio?
> 2. Se compilar, o que acontece quando você **executa**?
> 3. O sinal vai ser o **mesmo** do Passo A2? Por quê?

Compile:

```
gcc -Wall -Wextra -O0 -o privilegio privilegio.c
echo "gcc saiu com: $?"
```

```
gcc saiu com: 0
```

Nenhum aviso. De novo. E desta vez o silêncio é mais surpreendente que no Passo A2, porque lá o compilador não tinha como saber o valor do ponteiro — aqui a instrução privilegiada está escrita, literalmente, no código-fonte. Confira que ela foi mesmo parar no binário:

```
objdump -d privilegio | grep -B2 -A2 mrs
```

```
 840:   f9400000    ldr x0, [x0]
 844:   97ffff9b    bl  6b0 <fflush@plt>
 848:   d5381000    mrs x0, sctlr_el1
 84c:   f9000fe0    str x0, [sp, #24]
 850:   f9400fe1    ldr x1, [sp, #24]
```

Lá está ela, no endereço `848`, codificada como `d5381000`. (Os endereços da sua saída podem diferir; a codificação `d5381000` não.) Execute:

```
./privilegio
echo "codigo de saida: $?"
```

```
antes:  processo em modo usuario, prestes a ler SCTLR_EL1
Illegal instruction (core dumped)
codigo de saida: 132
```

A linha `depois:` nunca aparece. E o `132` segue a mesma regra do Passo A2 — **128 + número do sinal**:

```
kill -l 4
```

```
ILL
```

132 − 128 = 4 = SIGILL, de *illegal instruction*.

O endereço que o processador guardou

```
strace -e trace=write ./privilegio
```

```
write(1, "antes:  processo em modo usuario"..., 58) = 58
--- SIGILL {si_signo=SIGILL, si_code=ILL_ILLOPC, si_addr=0xaaaae6cd0848} ---
+++ killed by SIGILL +++
```

Mesma estrutura do Passo A2: uma única chamada de sistema (a `write` do `printf`), nenhuma outra, e depois o sinal. Mas olhe o `si_addr` com atenção:

```
si_addr=0xaaaae6cd0848
                  ^^^
```

Termina em `848` — exatamente o endereço onde o `objdump` mostrou a instrução `mrs`. O processador não só recusou a instrução: ele **registrou qual instrução recusou**, e o núcleo repassou esse endereço junto com o sinal. É a mesma contabilidade que no Passo A2 apontou o `si_addr=NULL`.

O que você acabou de descobrir

O compilador não sabe nada sobre privilégio. `mrs x0, sctlr_el1` é uma instrução ARM perfeitamente válida — ela existe, tem opcode, e o `gcc` a montou sem hesitar. O que ela não é é *sua*.

Quem verifica é o processador, em tempo de execução, a cada instrução. O bit que diz "o código que está executando agora está em modo usuário" mora na CPU, não no arquivo. É por isso que não existe verificação possível na compilação: **o mesmo binário, byte por byte, seria legal se executasse em modo núcleo.** A instrução não é ilegal; ela é ilegal *para você, agora*.

E é isso que fecha a Parte A. Você não pode entrar no modo núcleo por conta própria. As únicas maneiras de o processador acabar executando código privilegiado por causa do seu programa são as que você já viu: **pedir** (A1) ou **ser arrastado por uma exceção** (A2, A3). Em nenhuma delas quem executa o código privilegiado é você — é sempre o núcleo.

> **ATENÇÃO**
>
> ⚠ **Se `./privilegio` não morreu e imprimiu um valor**, então o código não estava em EL0 — anote o resultado e a configuração da sua máquina. É um resultado, não um erro do roteiro.

As três travessias da Parte A, lado a lado

	A1 — chamada de sistema	A2 — exceção por acesso	A3 — exceção por instrução
quem começou	o programa	o processador	o processador
foi voluntário?	sim — o programa pediu	não — o programa nem soube	não — mas o programa tentou
a instrução em si é ilegal?	não, é a instrução dedicada a pedir	não, é um armazenamento comum	**não** — é ilegal só neste nível

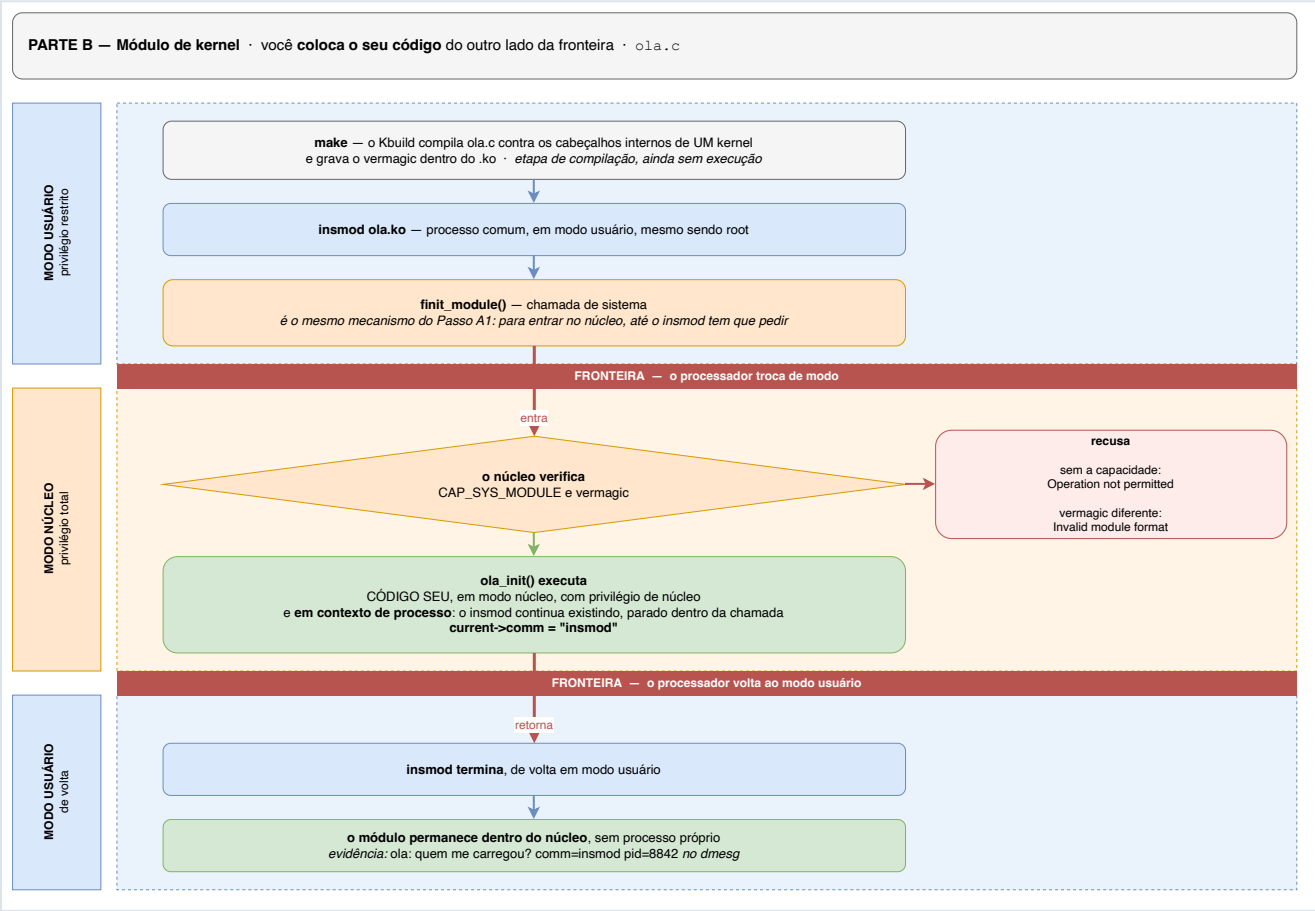
	A1 — chamada de sistema	A2 — exceção por acesso	A3 — exceção por instrução
o `gcc` avisou?	não há o que avisar	não	**não**, nem com `-Wall -Wextra`
mecanismo	chamada de sistema	exceção	exceção
o que aparece no `strace`	`read(3, …) = 13`	`--- SIGSEGV … ---`	`--- SIGILL … ---`
código de saída	0	139 = 128 + 11	132 = 128 + 4
o processo continua?	sim, na instrução seguinte	aqui não; em outros casos, sim	não
onde executa o código do núcleo	modo núcleo	modo núcleo	modo núcleo

A última linha é a que interessa. Três caminhos completamente diferentes, o mesmo destino: código do núcleo executando em modo privilegiado. E, nos três casos, **o processo continuou existindo** enquanto isso — ele não virou o núcleo, ele ficou parado esperando (A1) ou foi julgado por ele (A2, A3).

E é exatamente essa a ponte para a Parte B. Se do lado de cá não há como executar código privilegiado — o A3 provou —, mas existem momentos em que código do núcleo executa por causa do meu processo — o A1 provou —, então a única pergunta que sobra é: **e se esse código do núcleo fosse meu?**

PARTE B — Como colocar código seu dentro do núcleo

Escrever, compilar e tentar carregar **código que executa em modo núcleo** — o mesmo nível de privilégio do resto do kernel Linux. São 24 linhas de C que não têm `main()`, não viram executável e não podem usar a biblioteca C.



Fluxo da Parte B — carga de um módulo de kernel

Passo 1 — Leia o código antes de compilar

O arquivo `ola.c` já está nesta pasta. Abra e leia:

```c
#include <linux/module.h>   /* MODULE_*, module_init/module_exit */
#include <linux/kernel.h>   /* printk, KERN_INFO                 */
#include <linux/init.h>     /* __init, __exit                    */
#include <linux/sched.h>    /* current                           */

MODULE_LICENSE("GPL");                              /* ❶ */
MODULE_AUTHOR("Turma de SO - TADS/IFRN");
MODULE_DESCRIPTION("Prova de que estamos dentro do kernel");

static int __init ola_init(void)                    /* ❷ */
{
    printk(KERN_INFO "ola: carregado - estou em modo nucleo!\n");   /* ❸ */
    printk(KERN_INFO "ola: quem me carregou? comm=%s pid=%d\n",
           current->comm, current->pid);                            /* ❹ */
    return 0;                                                       /* ❺ */
}

static void __exit ola_exit(void)
{
    printk(KERN_INFO "ola: removido. Ate mais.\n");
}

module_init(ola_init);
module_exit(ola_exit);
```

Os cinco pontos que interessam

❶ `MODULE_LICENSE("GPL")` **não é burocracia.** O kernel *verifica* essa linha na carga. Sem uma licença compatível com a GPL o módulo até carrega, mas o kernel é marcado como **tainted** e centenas de símbolos `EXPORT_SYMBOL_GPL` ficam invisíveis para ele.

❷ **Não existe** `main()`**.** Um módulo é uma **biblioteca que o kernel liga a si mesmo**: `module_init()` aponta a função de entrada e `module_exit()` a de saída. A marca `__init` avisa que a função só serve uma vez — depois da carga o kernel **devolve a memória do código dela**.

❸ `printk`**, não** `printf`**.** A libc **não existe** deste lado da fronteira: nada de `printf`, `malloc` ou `fopen`. Aqui são `printk`, `kmalloc` e `kfree`. Repare no que isso significa para a Parte A: aquele `read()` que você chamou era uma função da biblioteca C pedindo um serviço; do lado de dentro **não há a quem pedir** — você *é* o serviço.

❹ `current` aponta para o processo em cujo **contexto** o código está executando **agora**. Ele é o fio que liga a Parte B à Parte A, e vale um bloco só para ele, logo abaixo.

❺ **O retorno importa.** Devolver diferente de zero faz o kernel **abortar a carga** e desfazer tudo.

O que `current` responde

Um mal-entendido natural depois da Parte A é imaginar que, ao entrar em modo núcleo, "o processo some" e o núcleo passa a rodar sozinho, à parte. Não é isso.

Quando você executa `insmod ola.ko`, o `insmod` é um processo comum, em modo usuário, e faz uma **chamada de sistema** (`finit_module`) — o mecanismo do passo A1, sem nenhuma novidade. Essa chamada leva o processador ao modo núcleo, o núcleo carrega o módulo e **chama a sua função** `ola_init()`. Nesse instante:

- o código que executa é **seu**, e executa em **modo núcleo**;

- o processo `insmod` **continua existindo**, parado dentro da chamada de sistema;
- é o **contexto dele** que está em uso — a pilha de núcleo dele, os dados de contabilidade dele.

É isso que se chama executar **em contexto de processo**. E é isso que `current` deixa medir: `current->comm` é o nome do processo em cujo contexto você está, e `current->pid` é o número dele. Você vai ver `comm=insmod`.

```
printk(KERN_INFO "ola: quem me carregou? comm=%s pid=%d\n",
       current->comm, current->pid);
```

Guarde a formulação certa: **entrar em modo núcleo não significa que deixou de existir um processo.** Significa que o processador passou a executar código privilegiado — e, neste caso, ainda por conta de um processo identificável.

> Nem todo código do núcleo executa assim. O tratador de uma **interrupção** de disco, por exemplo, roda quando o dispositivo resolve avisar, atropelando qualquer processo que estivesse no processador — e aí `current` não quer dizer "quem pediu isso". Guarde só a distinção; ela volta na Unidade 2, junto com o **PCB**, a estrutura onde o núcleo guarda tudo o que sabe sobre cada processo. `current` é um ponteiro para ela.

> **PREVEJA ANTES DE SEGUIR**
>
> > *Este arquivo tem `#include <linux/module.h>`, `printk()` e nenhuma `main()`. **O `gcc` compila isso?** Escreva sim ou não, e por quê.*

Passo 2 — O reflexo errado

Dentro do container:

```
cd /so/Aula03/codigo
gcc ola.c -o ola
```

```
ola.c:18:10: fatal error: linux/init.h: No such file or directory
   18 | #include <linux/init.h>     /* __init, __exit                    */
      |          ^~~~~~~~~~~~~~
compilation terminated.
```

Olhe com atenção qual linha falhou. Não foi a 16 (`linux/module.h`) nem a 17 (`linux/kernel.h`) — foi a **18**. Ou seja: o `gcc` **encontrou** os dois primeiros e só tropeçou no terceiro. Investigue:

```
ls /usr/include/linux/module.h /usr/include/linux/kernel.h /usr/include/linux/init.h
```

```
ls: cannot access '/usr/include/linux/init.h': No such file or directory
/usr/include/linux/kernel.h
/usr/include/linux/module.h
```

Então `linux/module.h` existe mesmo. Abra:

```
cat /usr/include/linux/module.h
```

```
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
#ifndef _LINUX_MODULE_H
#define _LINUX_MODULE_H

/* Flags for sys_finit_module: */
#define MODULE_INIT_IGNORE_MODVERSIONS  1
#define MODULE_INIT_IGNORE_VERMAGIC 2
#define MODULE_INIT_COMPRESSED_FILE 4

#endif /* _LINUX_MODULE_H */
```

Dez linhas, três `#define`, nenhum `MODULE_LICENSE`. Não é o `module.h` do kernel: é a versão **UAPI**, a fatia da interface do kernel publicada para o *espaço de usuário* — para quem vai **chamar** `finit_module()`, não para quem vai **ser** um módulo.

Note a coincidência que fecha a Parte A: esses são justamente os cabeçalhos de quem faz **chamadas de sistema**. Foi essa a família de arquivos que o `chamada.c` usou.

```
dpkg -S /usr/include/linux/module.h
```

```
linux-libc-dev:arm64: /usr/include/linux/module.h
```

O que você acabou de descobrir

Mesmo nome, conteúdo diferente, propósito oposto. Os cabeçalhos internos do kernel **não ficam em `/usr/include`** — eles vivem numa árvore separada, específica de **uma versão** de kernel. E não é só questão de achar os arquivos: o módulo precisa ser compilado com as mesmas *flags*, o mesmo alinhamento de estruturas e as mesmas opções de configuração com que aquele kernel foi construído.

É por isso que existe um Makefile nesta pasta.

Passo 3 — Então use o Makefile do kernel

```
make
```

```
ERRO: nenhuma árvore de cabeçalhos encontrada em /lib/modules/*/build.
      Rode:  apt-get update && apt-get install -y linux-headers-generic
make: *** [Makefile:25: all] Error 1
```

Confirme que a pasta realmente não existe:

```
ls /lib/modules/
```

```
ls: cannot access '/lib/modules/': No such file or directory
```

Aproveite e teste as ferramentas de módulo:

```
lsmod
```

```
bash: lsmod: command not found
```

Faltam as duas coisas. Instale:

```
apt-get update && apt-get install -y kmod linux-headers-generic
```

- **`kmod`** — traz `lsmod`, `modinfo`, `insmod`, `rmmod`, `modprobe`.
- **`linux-headers-generic`** — a árvore de cabeçalhos internos do kernel.

Custa **~8 segundos e 24 MB**. O container é descartável (`--rm`), então **isto se perde quando você sai** — ao voltar, rode de novo.

🔮 Preveja antes de seguir

Agora rode os dois comandos abaixo e **compare as duas saídas**:

```
ls /lib/modules/          # a versão dos cabeçalhos que você instalou
uname -r                  # a versão do kernel que está rodando agora
```

```
6.8.0-139-generic
6.12.76-linuxkit
```

> ***São diferentes.*** *Anote os dois números. Você acha que isso vai dar problema? Guarde a previsão — ela é cobrada no passo 7.*

Passo 4 — Compilar

```
make
```

```
==> kernel em execução : 6.12.76-linuxkit
==> compilando contra  : /lib/modules/6.8.0-139-generic/build
make -C /lib/modules/6.8.0-139-generic/build M=/so/Aula03/codigo modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-139-generic'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: aarch64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  You are using:           gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  CC [M]  /so/Aula03/codigo/ola.o
  MODPOST /so/Aula03/codigo/Module.symvers
  CC [M]  /so/Aula03/codigo/ola.mod.o
  LD [M]  /so/Aula03/codigo/ola.ko
Skipping BTF generation for /so/Aula03/codigo/ola.ko due to unavailability of vmlinux
  BTF [M] /so/Aula03/codigo/ola.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-139-generic'
```

> **O número da versão dos cabeçalhos muda.** Este roteiro foi medido com `6.8.0-139-generic`; o `apt` vai te entregar o que estiver publicado no dia. O que **não** muda é o fato que interessa: ele é diferente do kernel em execução.

Funcionou: você tem um **`ola.ko` de verdade**, de cerca de 270 KB.

> **Se você não viu as linhas `CC`, `MODPOST` e `LD`**, o módulo já estava construído de uma tentativa anterior e o `make` não teve o que fazer — ele só refaz o que mudou. Para ver a compilação inteira, force: `make clean && make`.

Repare no que o `make` fez, e por que o `gcc` sozinho não faria:

etapa	o que aconteceu
`make -C /lib/modules/…/build`	quem comanda é o **Makefile do kernel**, não o seu
`CC [M] ola.o`	compila o seu código com as flags daquele kernel
`MODPOST`	resolve os símbolos do kernel e **gera o `ola.mod.c`**
`CC [M] ola.mod.o`	compila *esse* arquivo gerado
`LD [M] ola.ko`	liga tudo no módulo final

Seu Makefile só informou duas coisas: o alvo é `ola.o` (`obj-m += ola.o`) e o código está aqui (`M=$(PWD)`). Todo o resto veio da árvore do kernel.

> **ATENÇÃO**
>
> ⚠ **A indentação das receitas do Makefile tem que ser TAB.** Espaços produzem `missing separator. Stop.` — é o erro nº 1 de quem escreve Makefile pela primeira vez.

Os dois avisos são normais aqui: *compiler differs* porque o kernel Ubuntu foi montado com `aarch64-linux-gnu-gcc-13` e você está com `gcc-13` — a saída mostra os dois lado a lado; *Skipping BTF* porque o pacote de cabeçalhos não traz o `vmlinux` completo.

Passo 5 — O que o `.ko` carrega dentro dele

```
modinfo ola.ko
```

```
filename:       /so/Aula03/codigo/ola.ko
description:    Prova de que estamos dentro do kernel
author:         Turma de SO - TADS/IFRN
license:        GPL
srcversion:     90C2D8EE24DCE341B19662B
depends:
name:           ola
vermagic:       6.8.0-139-generic SMP preempt mod_unload modversions aarch64
```

`description`, `author` e `license` são as três macros `MODULE_*` que você escreveu. `depends` está vazio porque este módulo não usa símbolo de nenhum outro.

O campo novo é o `vermagic` — a versão exata do kernel para a qual o módulo foi compilado, gravada dentro do arquivo. Ponha lado a lado com o kernel em execução:

```
make info
```

```
kernel em execução : 6.12.76-linuxkit
vermagic do módulo : 6.8.0-139-generic SMP preempt mod_unload modversions aarch64
```

> **PREVEJA ANTES DE SEGUIR — ESCREVA A RESPOSTA**
>
> 1. O `insmod ola.ko` vai funcionar?
> 2. Se não funcionar, **qual mensagem** você espera ver?
> 3. Você está como `root` neste container (confira com `whoami`). Isso ajuda?

Escreva as três respostas antes de rodar o próximo comando. Elas valem nota no entregável 6.

Passo 6 — A primeira tentativa

```
whoami
insmod ola.ko
```

```
root
insmod: ERROR: could not insert module ola.ko: Operation not permitted
```

Provavelmente não é a mensagem que você previu. Você é `root` e mesmo assim deu *permissão negada* — e o erro não fala nada sobre versão de kernel.

Investigue por que o `root` não bastou:

```
grep CapEff /proc/self/status
```

```
CapEff: 00000000a80c25fb
```

Esse número é um mapa de bits: cada bit é uma **capacidade**, um pedaço do poder que historicamente era tudo-ou-nada do `root`. Carregar módulo exige o bit **16** (`CAP_SYS_MODULE`). Verifique se ele está ligado:

```
C=$(grep CapEff /proc/self/status | cut -f2)
echo "CAP_SYS_MODULE (bit 16) = $(( 0x$C >> 16 & 1 ))"
```

```
CAP_SYS_MODULE (bit 16) = 0
```

O que você acabou de descobrir

`root` é permissão de software; capacidade é o que o kernel realmente confere. O Docker entrega ao container um `root` com a lista de capacidades **podada** — e `CAP_SYS_MODULE` ficou de fora de propósito.

E faz sentido: um módulo tem poder total sobre a máquina. Um container que pudesse carregar módulos **não isolaria nada** — derrubaria o hospedeiro e todos os outros containers junto. É a mesma lição de "*módulo não tem isolamento*" da aula, agora vista de fora para dentro.

> **`root` não é modo núcleo.** Volte ao vocabulário do começo do roteiro: `root` é uma identidade, verificada por software pelo próprio sistema operacional, que autoriza certas operações. Modo usuário/núcleo é uma propriedade do **contexto de execução no processador**. O processo `insmod`, mesmo sendo do `root`, executou o próprio código em modo usuário do começo ao fim — e a única forma que ele teve de alcançar o núcleo foi a do passo A1: pedindo, por chamada de sistema. O que a capacidade decide é se aquele pedido **é atendido**.

E repare: essa barreira não diz nada sobre o `vermagic`. A sua previsão do passo 5 pode estar certa e ainda não ter sido testada — a permissão barrou antes.

Passo 7 — Removendo a primeira barreira

Vamos dar a capacidade que faltava e ver o que aparece atrás dela. **Saia do container** (`exit`) e suba outro, igual, com uma flag a mais:

```
# na sua máquina, a partir da pasta ambiente/ — NÃO dentro do container
docker run --rm -it -v "$PWD":/so -w /so/Aula03/codigo --cap-add=SYS_MODULE so-ifrn bash
```

Lá dentro, repita a instalação e a compilação:

```
apt-get update && apt-get install -y kmod linux-headers-generic
make
```

Confirme que agora a capacidade está lá:

```
C=$(grep CapEff /proc/self/status | cut -f2)
echo "CAP_SYS_MODULE (bit 16) = $(( 0x$C >> 16 & 1 ))"
```

```
CAP_SYS_MODULE (bit 16) = 1
```

E tente de novo:

```
insmod ola.ko
```

```
insmod: ERROR: could not insert module ola.ko: Invalid module format
```

A mensagem mudou. Não é mais permissão — agora é aquela diferença de versão que você anotou no passo 3 e previu no passo 5.

Por que o kernel é tão rígido

O módulo é **ligado ao kernel** e enxerga estruturas internas dele — o *layout* de `struct task_struct`, aquela mesma para onde `current` aponta — que mudam de versão para versão. Se o kernel aceitasse carregar assim, o seu código leria campos nos deslocamentos errados: **corrupção silenciosa de memória em modo núcleo**. Nenhum erro, nenhum aviso, e a máquina corrompendo dados até travar.

Compare com o que aconteceu na Parte A2, e a diferença é gritante: lá, um acesso inválido em modo usuário foi detectado pelo processador e o processo morreu sozinho, sem incomodar mais ninguém. Aqui não haveria detecção nenhuma — o código estaria em modo núcleo, onde nada o vigia. Recusar a carga é a única defesa possível, e o `vermagic` é como o kernel detecta.

> **É a mesma dor do driver da NVIDIA.** Toda vez que o Ubuntu atualiza o kernel, o driver precisa ser **recompilado** — e é exatamente isso que o **DKMS** automatiza.

Nesta máquina não há como resolver: ninguém publica cabeçalhos para o kernel `linuxkit` do Docker Desktop, então não existe como compilar para o kernel que está rodando. Para carregar de verdade, veja o **passo 10**.

Passo 8 — Síntese: as duas barreiras

Você atravessou duas, e elas são independentes:

barreira	erro	natureza	como se remove
1ª — permissão	`Operation not permitted`	o container não tem `CAP_SYS_MODULE`	`--cap-add=SYS_MODULE`
2ª — compatibilidade	`Invalid module format`	`vermagic` ≠ kernel em execução	recompilar **no** kernel de destino

E o que o container permite, afinal:

etapa	funciona aqui?
escrever o `.c`	✅
compilar o `.ko`	✅ (após instalar os cabeçalhos)
`modinfo`, `lsmod`, `/proc/modules`	✅
`insmod` / `rmmod`	❌ as duas barreiras acima
`dmesg`	❌ exige `CAP_SYSLOG`, também podada

Passo 9 — Limpar

```
make clean
ls *.ko *.o
```

```
ls: cannot access '*.ko': No such file or directory
ls: cannot access '*.o': No such file or directory
```

As duas mensagens são o resultado desejado: nenhum artefato de compilação sobrou. Faça isso antes de entregar.

Passo 10 — Carregando de verdade (fora do container)

Para ver o `printk` aparecer no `dmesg`, use um Linux cujo kernel seja o seu:

- **Windows** — WSL2 com Ubuntu, instalando `linux-headers-$(uname -r)`.
- **Linux nativo** — funciona direto.
- **VM (VirtualBox, UTM, VMware)** com Ubuntu — o caminho mais previsível.

```
sudo apt install build-essential linux-headers-$(uname -r)
cd Aula03/codigo
make
sudo insmod ola.ko            # agora carrega
lsmod | grep ola
sudo dmesg | tail -2
sudo rmmod ola
sudo dmesg | tail -1
```

Saída esperada:

```
$ lsmod | grep ola
ola                    12288  0

$ sudo dmesg | tail -2
[ 9421.551] ola: carregado - estou em modo nucleo!
[ 9421.551] ola: quem me carregou? comm=insmod pid=8842

$ sudo rmmod ola
$ sudo dmesg | tail -1
[ 9455.108] ola: removido. Ate mais.
```

`comm=insmod` — o fecho da prática

Esta linha é o ponto para o qual o roteiro inteiro foi construído. Releia o percurso:

```
insmod (processo, modo usuário)
        ↓  chamada de sistema finit_module()   ← o mecanismo do passo A1
     modo núcleo
        ↓  o núcleo carrega o módulo
     ola_init()  ← código SEU, em modo núcleo,
                   no contexto do processo insmod
        ↓  printk registra comm=insmod pid=8842
     retorno
        ↓
insmod termina, em modo usuário
```

O módulo **não tem processo próprio** e não tem `main()`: ninguém o "executa". Ele é código que fica disponível dentro do núcleo, e a sua função de inicialização foi chamada **no contexto do processo que pediu a**

carga. Por isso `current` respondeu `insmod`, e por isso o `pid` é o do `insmod` — confira com `pgrep`, se quiser, embora ele já tenha terminado.

O que você fez: adicionou ~12 KB de código seu ao núcleo de um sistema operacional monolítico **com a máquina ligada**, e removeu depois. É a modularidade do capítulo *Monolítico* — e a fragilidade também: um ponteiro errado nessas 24 linhas não daria `Segmentation fault` como no passo A2. Daria *kernel panic*, porque do lado de dentro **não há um núcleo acima para detectar a falta e matar só o culpado**.

PARTE C — O que este experimento demonstrou

Responda mentalmente antes de ler: *o que é, afinal, a fronteira entre modo usuário e modo núcleo, e onde exatamente ela foi atravessada nesta prática?*

O quadro completo

#	conceito	onde apareceu	evidência que você viu
1	**modo usuário**	A1, A2, A3	os `printf`, o `*p = 42`, o `mrs`: código seu, executado pelo processador com privilégio restrito
2	**chamada de sistema**	A1	as linhas `read(3, …) = 13` no `strace` — uma por travessia, pedida pelo programa
3	**modo núcleo**	A1, A2, A3, B	onde o serviço de `read` foi executado, onde os sinais foram decididos, onde `ola_init()` rodou
4	**retorno ao modo usuário**	A1	o `= 13` da linha do `strace`: o valor já chegou de volta, e a execução seguiu na instrução seguinte
5	**exceção por acesso inválido**	A2	`--- SIGSEGV {si_code=SEGV_MAPERR, si_addr=NULL} ---` sem nenhuma chamada de sistema antes
6	**exceção por instrução privilegiada**	A3	`--- SIGILL {si_code=ILL_ILLOPC, si_addr=…848} ---`, com o `848` do `objdump`
7	**o que modo usuário não pode fazer**	A3	o `gcc` montou `mrs x0, sctlr_el1` em silêncio, e o processador recusou executá-la
8	**código de módulo em contexto de kernel**	B, passo 10	`ola: carregado – estou em modo nucleo!` no `dmesg`
9	**contexto de processo**	B, passo 10	`comm=insmod pid=8842` — havia um processo por trás do código de núcleo
10	**privilégio ≠ modo de execução**	B, passos 6 e 7	`whoami` → `root`, e mesmo assim `Operation not permitted`; bit 16 = 0

As seis frases que você deve conseguir defender

1. **Um processo não "está" em modo usuário ou em modo núcleo como quem está num cômodo.** Ele normalmente executa o próprio código em modo usuário; em certos momentos o processador transfere o controle para código do núcleo, que executa em modo privilegiado, e depois devolve.
2. **`read()` não é o núcleo.** É uma função da biblioteca C, no seu processo, em modo usuário, que emite a instrução de chamada de sistema. A prova é o `syscall(SYS_read, …)`: mesmo serviço, mesma linha no `strace`, sem a função.
3. **Nem toda entrada no núcleo é pedida.** A chamada de sistema é voluntária; a exceção é o processador constatando, no meio de uma instrução, que ela não pode ser concluída; a interrupção vem de um dispositivo externo, sem relação com a instrução em curso. Três mecanismos, um destino.

4. **Não dá para entrar no modo núcleo por conta própria.** A instrução privilegiada escrita no seu código compila sem um aviso e mesmo assim não executa: quem verifica o privilégio é o processador, a cada instrução, em tempo de execução. O mesmo binário seria legal em modo núcleo — a instrução não é ilegal, ela é ilegal *para o seu código*.
5. **`root` é identidade; modo núcleo é estado do processador.** O `insmod` era `root` e ainda assim executava em modo usuário, e ainda assim foi barrado — porque faltava `CAP_SYS_MODULE`, que é o kernel decidindo, por software, se aquele pedido é atendido.
6. **Um módulo é código do núcleo, com privilégio de núcleo.** Ele é carregado dentro do kernel e passa a executar com privilégios de kernel. Quando a sua função de inicialização roda por causa de um `insmod`, ela roda **em contexto de processo** — e é por isso que `current->comm` respondeu `insmod`.

Uma última comparação

	você pediu (A1)	você errou (A2)	você tentou (A3)	você entrou (B)
mecanismo	chamada de sistema	exceção	exceção	chamada de sistema (`finit_module`) + carga
quem decidiu	o programa	o processador	o processador	o kernel, pela capacidade e pelo `vermagic`
onde o seu código estava	modo usuário	modo usuário	modo usuário	**modo núcleo**
quem te barrou	ninguém	a MMU e o kernel, com um sinal	o próprio processador, com um sinal	`CAP_SYS_MODULE`, depois o `vermagic`
resultado	serviço prestado	processo morto	processo morto	módulo carregado (fora do container)

Nas três primeiras colunas a fronteira **funcionou a seu favor**: ela te deu serviços (A1), te protegeu do seu próprio erro (A2) e impediu que o seu código mexesse na máquina inteira (A3). Na quarta ela funcionou **contra** você — e sempre pelo mesmo motivo. É a mesma linha, vista dos dois lados.

Entregáveis

Um PDF ou Markdown com:

Da Parte A

1. **A saída do `strace -e trace=openat,read,lseek,close ./chamada > /dev/null`**, com as duas linhas de `read` do seu código destacadas. Em três a cinco linhas: por que elas são idênticas, se uma veio de `read()` e a outra de `syscall(SYS_read, …)`?
2. **A saída do `strace -e trace=write ./excecao`** e o código de saída do programa. Explique a conta `139 = 128 + 11` e responda: **quantas chamadas de sistema o programa fez para pedir o tratamento do erro?** Justifique a partir do que aparece — e do que não aparece — no rastro.
3. **A saída do `objdump` e do `strace` do `./privilegio`**, com o código de saída. Aponte, nas duas saídas, **o mesmo endereço** — o do `objdump` e o do `si_addr` — e explique em três linhas por que o `gcc` não

avisou nada.

4. **Uma tabela sua**, de três colunas, comparando A1, A2 e A3 em pelo menos cinco aspectos (quem começou, foi voluntário, se a instrução é ilegal em si, o que aparece no `strace`, o código de saída, o que acontece com o processo).

Da Parte B

5. **A saída do `gcc ola.c`** (passo 2) e a explicação: por que ele encontrou `linux/module.h` e mesmo assim não serviu? Cite o conteúdo do arquivo.
6. **As três previsões que você escreveu no passo 5**, seguidas das saídas reais dos passos 6 e 7. Para cada previsão: acertou ou errou, e o que faltava saber. *Errar aqui não tira nota — não registrar a previsão, sim.*
7. **A saída do `grep CapEff`** e a conta do bit 16, nas duas configurações. Explique em duas frases a diferença entre **ser `root`** e **ter `CAP_SYS_MODULE`** — e por que nenhuma das duas é a mesma coisa que "estar em modo núcleo".
8. **Se você chegou ao passo 10**: a saída do `dmesg` com as duas linhas do módulo, e uma explicação de por que aparece `comm=insmod` — mencionando o mecanismo do passo A1.
9. **Uma modificação sua.** Escolha uma:
   - fazer o `ola_init()` retornar `-EINVAL` e mostrar o que muda;
   - acrescentar um `printk` que imprima outro campo de `current` (ex.: `current->tgid`);
   - trocar `MODULE_LICENSE("GPL")` por `"Proprietary"` e relatar o que o kernel diz;
   - mudar o `chamada.c` para ler um arquivo que **não existe** e explicar, pelo `strace`, o que o núcleo devolveu e como a biblioteca C transformou isso no `perror`;
   - trocar a instrução privilegiada do `privilegio.c` por outra do mesmo nível e verificar se o sinal continua sendo `SIGILL`.

Da Parte C

10. **As previsões escritas** de todos os checkpoints 🔮, do jeito que você as escreveu **antes** de rodar, com uma linha em cada dizendo o que a saída real mostrou.

Questões

1. Um colega diz: "*a função `read()` executa no kernel*". Corrija a frase em no máximo três linhas, usando o que o `strace` mostrou no passo A1.
2. No passo A2 o `gcc -Wall -Wextra` não emitiu **nenhum** aviso para um programa que morre na terceira linha. Por que o compilador não tem como saber? *(A questão 5 faz a mesma pergunta sobre o passo A3 — e a resposta é outra. Compare as duas.)*
3. Explique a diferença entre **exceção** e **interrupção** usando um exemplo de cada. Qual das duas o `excecao.c` provocou, e como você sabe?
4. Os passos A2 e A3 usam **o mesmo mecanismo** — exceção — e entregam **sinais diferentes**. O que exatamente o processador detectou em cada um? Por que faz sentido o SO distinguir os dois casos com sinais diferentes, em vez de matar o processo do mesmo jeito nos dois?

5. No A3, o `gcc -Wall -Wextra` montou uma instrução privilegiada sem um único aviso. Um colega propõe: "*então é só o compilador passar a recusar essas instruções*". Diga por que isso não resolve — o que o compilador teria que saber, e que ele não pode saber?
6. Por que `gcc ola.c -o ola` não funciona, mesmo depois de instalar os cabeçalhos? O que o Makefile do kernel faz que o `gcc` sozinho não faria?
7. Um colega compilou o módulo no computador dele (kernel `6.8.0-45`) e mandou o `.ko` pronto para você (kernel `6.8.0-52`). Vai carregar? Qual mensagem você espera, e qual é a solução?
8. Se o Docker liberasse `CAP_SYS_MODULE` por padrão, que garantia de isolamento se perderia? Responda pensando em **outros** containers na mesma máquina.
9. O que significa dizer que um módulo carregado **não tem isolamento**? Compare com o que aconteceria num micronúcleo, onde o driver é um processo. Use o passo A2 na resposta: o que aconteceria se `*p = 42` estivesse dentro de `ola_init()`?
10. Depois do `insmod`, a memória da função `ola_init()` é devolvida ao sistema. Que marca no código autoriza o kernel a fazer isso, e por que ela não pode ser usada em `ola_exit()`?
11. `current->comm` respondeu `insmod`. Isso significa que o módulo "pertence" ao processo `insmod`? O que acontece com o módulo depois que o `insmod` termina?

Erros comuns

Consulte só depois de tentar entender a falha — várias delas são passos do roteiro.

Mensagem	Causa	Onde está tratada
`Segmentation fault` no `./excecao`	é o experimento	passo A2 — é intencional
`Illegal instruction` no `./privilegio`	é o experimento	passo A3 — é intencional
`./privilegio` imprimiu um valor e não morreu	o código não estava em EL0	anote e relate — é um resultado
`strace: Operation not permitted`	você não entrou pelo `so.sh`	o `strace` usa `ptrace(2)`, e o container só a recebe com `--cap-add=SYS_PTRACE`, que o `so.sh` já passa
`Antes do acesso` não apareceu	você redirecionou a saída e tirou o `fflush`	a libc usa buffer cheio quando a saída não é terminal, e o processo morre antes de esvaziá-lo
o `gcc` avisou `null pointer dereference`	você compilou com `-O2` ou mais	recompile com `-O0`. Com `-O2` o programa ainda morre com 139, mas o aviso aparece — e o silêncio do compilador é a descoberta do passo
`fatal error: linux/init.h`	usou `gcc` direto	passo 2 — é intencional
`nenhuma árvore de cabeçalhos encontrada`	cabeçalhos não instalados	passo 3 — é intencional
`lsmod: command not found`	falta o pacote `kmod`	passo 3

Mensagem	Causa	Onde está tratada
`Operation not permitted` no `insmod`	falta `CAP_SYS_MODULE`	passo 6 — é intencional
`Invalid module format`	`vermagic` ≠ kernel em execução	passo 7 — é intencional
`Operation not permitted` no `dmesg`	falta `CAP_SYSLOG`	passo 10
`missing separator. Stop.`	indentação com espaços no Makefile	troque por **TAB**
`make` não mostra `CC` / `LD`	já estava construído	`make clean && make`

Arquivos desta pasta

Os programas ficam todos em `codigo/`, que é onde você trabalha:

codigo/	o que é
`chamada.c`	**Parte A1** — pede um serviço ao núcleo, por dois caminhos
`excecao.c`	**Parte A2** — provoca uma exceção de acesso; morre de propósito
`privilegio.c`	**Parte A3** — executa uma instrução privilegiada; morre de propósito
`ola.c`	**Parte B** — o módulo, comentado com as cinco notas
`Makefile`	detecta a árvore de cabeçalhos e chama o Kbuild

E na pasta de cima:

Aula03/	o que é
`fluxo-fronteira.*`	o diagrama das quatro travessias, no começo deste roteiro
`fluxo-a1.*`, `fluxo-a2.*`, `fluxo-a3.*`, `fluxo-b.*`	o diagrama de cada passo, ampliado
`fronteira/`	**um laboratório à parte**, opcional: a mesma fronteira **medida** — custo de uma travessia em nanossegundos, contagem de chamadas com `strace -c`, dedução do tamanho do buffer da libc
`LEIAME.md`	este roteiro
`_ferramentas/`	scripts que geram os PDFs e os diagramas a partir deste Markdown — uso do professor

Referências

- **Tanenbaum, *Sistemas Operacionais Modernos*, 4ª ed.** — **§1.6, p. 35** (impressa), chamadas de sistema, com a **Figura 1.17** decompondo `read()` em 11 passos: os passos 1 a 3 e 9 a 11 são modo usuário; os 6 do meio são o núcleo. É literalmente o passo A1 desenhado. **§1.7, p. 43**, estrutura do SO — monolítico, módulos e micronúcleo.

- **Maziero, *Sistemas Operacionais: Conceitos e Mecanismos*** — §2.2, **níveis de privilégio, p. 20** (impressa), e **chamadas de sistema, p. 21**. É a descrição mais direta dos dois modos e da instrução que faz a transição. Atenção ao vocabulário: Maziero usa **núcleo** onde Tanenbaum usa *kernel*.
- `man 2 syscall`, `man 2 read`, `man 7 signal`, `man 1 strace`, `man 7 capabilities`.

Prática da Aula 03 · Sistemas Operacionais · Prof. Bonfim Aquino · IFRN — 2026 Todas as saídas deste roteiro foram verificadas na imagem `so-ifrn` (Ubuntu 24.04.4, kernel 6.12.76-linuxkit aarch64, GCC 13.3.0, Docker Desktop no macOS): a Parte B em 01/09/2026, os Passos A1 a A3 em 10/09/2026.